

Bài 1:

1. OpenWeather

Loại API: REST

Base URL: <https://api.openweathermap.org>

Auth method: API Key, truyền qua tham số (?appid=YOUR_API_KEY)

Versioning: Có versioning, tùy thuộc vào endpoint (/data/2.5/ hoặc data/3.0/)

Resource identifier: City ID (Dạng số) hoặc toạ độ (latitude/longitude)

2. ExchangeRate-API

Loại API: REST

Base URL: <https://v6.exchangerate-api.com/v6/>

Auth method: API Key, thông qua URL (/v6/YOUR_API_KEY)

Versioning: Có versioning, thông qua URL (/v6/)

Resource identifier: Mã tiền tệ dạng chuỗi ký tự

3. Spotify Web API

Loại API: REST

Base URL: <https://api.spotify.com/v1/>

Auth method: Access Token theo tiêu chuẩn OAuth 2.0

Versioning: Có versioning, thông qua URL (/v1/)

Resource identifier: Chuỗi Base-62

Bài 2:

Các đặc điểm của một API tốt theo JJ Geewax:

- Operational: API phải thực hiện được chức năng người dùng cần
- Expressive: API cần biểu đạt rõ ràng để người sử dụng hiểu được API đang cung cấp chức năng gì
- Simple: API nên làm cho các use case phổ biến trở nên đơn giản
- Predictable: API nên hoạt động nhất quán

Các đặc điểm của một API tốt theo James Higginbotham:

- Thiết kế API cần có sự tham gia của nhiều bên
- Thiết kế API phải bắt đầu từ outcome
- Chọn loại API phù hợp theo nhu cầu
- Documentation là UI quan trọng nhất với Dev
- API cần được thiết kế để thích nghi với sự thay đổi

Các đặc điểm trong Slide 05 chủ yếu nói về đặc tính kỹ thuật của API, trong khi đó Geewax và Higginbotham bổ sung thêm khía cạnh về khả năng sử dụng, giá trị business và vòng đời API.

Bài 3:

a.

```
q = request.args.get('q', "").strip().lower()
if q:
    books = [b for b in books if q in b['title'].lower() or q in b['author'].lower()]
```

b.

```
sort = request.args.get('sort')
if sort:
    if sort != 'title':
        return jsonify({'error': 'unsupported sort field'}), 400
    books = sorted(books, key=lambda b: b['title'].lower())
```

c.

```
def validate_year(year):
    return isinstance(year, int) and not isinstance(year, bool) and year >= 1900
```

Trong hàm create_book():

```
y = body.get('year')
if not validate_year(y):
    return jsonify({'error': 'invalid year'}), 400
```

Trong hàm modify_book():

```
if request.method == 'PUT':
    body = request.get_json(silent=True) or {}

    if 'year' in body and not validate_year(body['year']):
        return jsonify({'error': 'invalid year'}), 400

    book.update(body)
    return jsonify(book), 200
```